

CUDA 转行 Day 1 → Day 180 学习计划

目标：从非计算机科班基础，逐步进入 GPU / CUDA / AI Systems / Inference 工程方向

学习强度：每天约 3 小时 | 周期：180 天 | 核心产出：GitHub 作品集 + 性能优化能力

这不是“刷完课程”的计划，而是一条以项目、Benchmark、Profiler 和 GitHub 为核心的实战路线。每天建议 45 分钟理论 + 90 分钟编码 + 30 分钟性能测试/Profiler + 15 分钟笔记与 Git commit。

最终目标不是成为“只会 CUDA 的程序员”，而是逐步形成 GPU Performance / AI Kernel / AI Inference / AI Systems Engineer 的能力栈。

180 天路线总览

阶段	天数	核心内容	主要产出
0	1 – 30	C++ / Linux / Git / CMake / GPU 基础	CPU Matrix + C++ 基础仓库
1	31 – 60	CUDA 编程模型 / Memory / Warp	Vector Add / Matrix Add / Naive GEMM
2	61 – 90	CUDA 性能优化 / Nsight	优化 GEMM / Reduction / Transpose
3	91 – 120	PyTorch / CUDA Extension / AI Kernel	Softmax / LayerNorm / RMSNorm
4	121 – 150	Triton / Attention	CUDA vs Triton Kernel
5	151 – 180	FlashAttention / Portfolio	旗舰项目 + 完整 GitHub 作品集

GitHub / 官方资源

NVIDIA CUDA Samples — <https://github.com/NVIDIA/cuda-samples> — CUDA 官方示例库。

CUDA Programming Guide — <https://docs.nvidia.com/cuda/cuda-programming-guide/> — CUDA 主教材。

Triton — <https://github.com/triton-lang/triton> — Day 121 后重点学习。

PyTorch Custom C++/CUDA Operators — https://docs.pytorch.org/tutorials/advanced/cpp_custom_ops.html — Day 105 后重点学习。

NVIDIA CUTLASS — <https://github.com/NVIDIA/cutlass> — 180 天后继续深入 Tensor Core / GEMM。

第 1 周 | Day 1 – 7

重点：Git / Linux / C++ 入门

- Day 1：安装 Git、VS Code、CMake、C++；创建 cuda-career-180；完成 hello.cpp 与第一次 commit。
- Day 2：C++ 基础类型、变量、输入输出；完成 hello.cpp、calculator.cpp。
- Day 3：if / for / while / switch；完成 prime.cpp、fibonacci.cpp。
- Day 4：函数、reference、const；完成 vector_sum.cpp。
- Day 5：pointer、地址、解引用；理解 int x / int* p。
- Day 6：数组、指针运算；完成 array_sum.cpp。
- Day 7：周测：不看资料完成 array sum / max / reverse array。

第 2 周 | Day 8 – 14

重点：C++ 面向对象 + STL

- Day 8：struct / class。
- Day 9：constructor / destructor / this。
- Day 10：public / private / protected。
- Day 11：STL vector。
- Day 12：string / array。
- Day 13：unordered_map。
- Day 14：周测：实现 Matrix 类，支持 add / multiply / print。

第 3 周 | Day 15 – 21

重点：现代 C++ 核心

- Day 15：reference / const reference。
- Day 16：pointer vs reference。
- Day 17：unique_ptr / shared_ptr。
- Day 18：RAII。
- Day 19：template。
- Day 20：lambda。
- Day 21：周测：template Matrix。

第 4 周 | Day 22 – 30

重点：CMake / GDB / Linux / CPU Benchmark

- Day 22：CMakeLists.txt。
- Day 23：GDB：break / run / next / print。
- Day 24：Linux：ps / top / htop / grep / find / sed / awk。
- Day 25：cat / less / tail / head。
- Day 26：理解 source → preprocess → compile → assembly → link。
- Day 27：比较 -O0 / -O2 / -O3。
- Day 28：static / dynamic / shared library。

- Day 29：实现 CPU Matrix Multiplication。
- Day 30：月考：Matrix + Matrix Multiplication + Benchmark + CMake + README。

第 5 周 | Day 31 – 37

重点：CUDA 编程模型

- Day 31：CPU vs GPU。
- Day 32：Thread / Block / Grid。
- Day 33：第一个 `__global__` Kernel。
- Day 34：`cudaMalloc` / `cudaMemcpy` / `cudaFree`。
- Day 35：`cudaDeviceSynchronize`。
- Day 36：nvcc 编译。
- Day 37：Vector Add。

第 6 周 | Day 38 – 44

重点：CUDA Indexing + Matrix

- Day 38：`threadIdx` / `blockIdx` / `blockDim` / `gridDim`。
- Day 39：二维 Grid。
- Day 40：二维 Matrix。
- Day 41：Matrix Add。
- Day 42：Naive Matrix Multiplication。
- Day 43：CPU vs CUDA Benchmark。
- Day 44：README 记录 GPU、输入规模、CPU 时间、GPU 时间、Speedup。

第 7 周 | Day 45 – 51

重点：GPU Memory

- Day 45：Global Memory。
- Day 46：Register。
- Day 47：Shared Memory。
- Day 48：Constant Memory。
- Day 49：Memory Hierarchy。
- Day 50：Memory Bandwidth。
- Day 51：实现 `memory_bandwidth` benchmark。

第 8 周 | Day 52 – 60

重点：Warp + CUDA Samples

- Day 52：Warp。
- Day 53：Warp Divergence。
- Day 54：SIMT。
- Day 55：Branch 行为。
- Day 56：branch benchmark。

- Day 57：阅读 cuda-samples：simple / reduction / transpose。
- Day 58 – 59：重新独立实现 vector_add / matrix_add。
- Day 60：考试：从零写 Vector Add、Matrix Add、Matrix Multiply，并解释每个 thread 的工作。

第 9 周 | Day 61 – 67

重点：Coalescing + Shared Memory

- Day 61：Coalesced Memory Access。
- Day 62：coalesced.cu。
- Day 63：non_coalesced.cu。
- Day 64：Benchmark。
- Day 65：Shared Memory。
- Day 66：Shared-memory Tiling。
- Day 67：优化 Matrix Multiplication。

第 10 周 | Day 68 – 74

重点：GEMM 优化

- Day 68：Naive GEMM。
- Day 69：Shared-memory GEMM。
- Day 70：测试 tile size 8 / 16 / 32。
- Day 71：Register Blocking。
- Day 72：Loop Unrolling。
- Day 73：Benchmark。
- Day 74：README 记录 Naive → Shared → Tiled → Register。

第 11 周 | Day 75 – 81

重点：Reduction

- Day 75：Reduction 原理。
- Day 76：Naive Reduction。
- Day 77：Shared-memory Reduction。
- Day 78：Warp Reduction。
- Day 79：__shfl_down_sync。
- Day 80：Atomic。
- Day 81：优化 Reduction。

第 12 周 | Day 82 – 90

重点：Transpose + Nsight

- Day 82：Transpose。
- Day 83：Shared-memory Transpose。
- Day 84：Bank Conflict。
- Day 85：解决 Bank Conflict。

- Day 86 : Benchmark。
- Day 87 : Profiler 基础。
- Day 88 : Nsight Systems。
- Day 89 : Nsight Compute。
- Day 90 : 考试 : 用 profiler 找到至少一个 kernel bottleneck, 并解释为什么慢。

第 13 周 | Day 91 – 97

重点 : PyTorch + Softmax

- Day 91 : Tensor / Matrix / Vector。
- Day 92 : PyTorch CUDA Tensor。
- Day 93 : torch.cuda。
- Day 94 : PyTorch vs 自写 CUDA Benchmark。
- Day 95 : Softmax 数学。
- Day 96 : CUDA Softmax。
- Day 97 : Benchmark。

第 14 周 | Day 98 – 104

重点 : LayerNorm / RMSNorm

- Day 98 : LayerNorm 数学。
- Day 99 : CUDA LayerNorm。
- Day 100 : 优化 LayerNorm。
- Day 101 : RMSNorm。
- Day 102 : CUDA RMSNorm。
- Day 103 : Benchmark。
- Day 104 : 整理 README。

第 15 周 | Day 105 – 111

重点 : PyTorch C++ / CUDA Extension

- Day 105 : PyTorch C++ Extension。
- Day 106 : torch.utils.cpp_extension。
- Day 107 : C++ Operator。
- Day 108 : CUDA Operator。
- Day 109 : PyTorch 调用 CUDA。
- Day 110 : Correctness Test。
- Day 111 : Benchmark。

第 16 周 | Day 112 – 120

重点 : PyTorch Custom Operator 完整化

- Day 112 – 115 : torch.library、TORCH_LIBRARY、FakeTensor、opcheck。
- Day 116 : 完成 pytorch-cuda-extension。

- Day 117：补测试。
- Day 118：Benchmark。
- Day 119：README。
- Day 120：考试：完整跑通 Python → PyTorch → C++ Extension → CUDA → GPU。

第 17 周 | Day 121 – 127

重点：Triton 入门

- Day 121：安装 Triton。
- Day 122：第一个 Triton Kernel。
- Day 123：program_id。
- Day 124：tl.load / tl.store。
- Day 125：Mask。
- Day 126：Vector Add。
- Day 127：Benchmark。

第 18 周 | Day 128 – 134

重点：Triton AI Kernel

- Day 128：Triton Softmax。
- Day 129：优化 Softmax。
- Day 130：CUDA vs Triton。
- Day 131：Triton LayerNorm。
- Day 132：Triton RMSNorm。
- Day 133：Benchmark。
- Day 134：整理 cuda-vs-triton。

第 19 周 | Day 135 – 141

重点：Triton Matmul

- Day 135：Triton Matmul。
- Day 136：Tiling。
- Day 137：Block Size。
- Day 138：Autotuning。
- Day 139：Benchmark。
- Day 140：阅读 Triton Tutorials。
- Day 141：总结 CUDA vs Triton。

第 20 周 | Day 142 – 147

重点：Attention

- Day 142：Q / K / V。
- Day 143： QK^T 。
- Day 144：Softmax。

- Day 145 : Attention = softmax(QK^T)V。
- Day 146 : Naive Attention。
- Day 147 : CUDA Attention。

第 21 周 | Day 148 – 150

重点：Memory-efficient Attention

- Day 148 : Memory-efficient Attention。
- Day 149 : IO Complexity。
- Day 150 : 考试：画出 $Q \rightarrow QK^T \rightarrow \text{Softmax} \rightarrow \times V$ ，并解释普通 Attention 的中间矩阵开销。

第 22 周 | Day 151 – 157

重点：FlashAttention 第一版

- Day 151 : 研究 FlashAttention。
- Day 152 : Tiling。
- Day 153 : Online Softmax。
- Day 154 : 实现 FlashAttention v0。
- Day 155 : Correctness。
- Day 156 : Benchmark。
- Day 157 : 第一轮优化。

第 23 周 | Day 158 – 164

重点：Profiler + 优化

- Day 158 : Nsight Compute。
- Day 159 : 分析 Memory / Warp / Occupancy。
- Day 160 : 第二版优化。
- Day 161 : 与 PyTorch 对比。
- Day 162 : 与 Triton 对比。
- Day 163 : 整理 Benchmark。
- Day 164 : README。

第 24 周 | Day 165 – 171

重点：旗舰项目包装

- Day 165 : 画 Architecture Diagram。
- Day 166 : 代码清理。
- Day 167 : 补测试。
- Day 168 : GitHub Release。
- Day 169 : 创建 GPU Performance Lab。
- Day 170 : 统一 PyTorch / CUDA / Triton Benchmark。
- Day 171 : Vector Add Benchmark。

第 25 周 | Day 172 – 178

重点：作品集

- Day 172：Softmax Benchmark。
- Day 173：RMSNorm Benchmark。
- Day 174：Matmul Benchmark。
- Day 175：Attention Benchmark。
- Day 176：生成 benchmark.csv 与图表。
- Day 177：整理 GitHub。
- Day 178：补齐项目文档与运行说明。

第 26 周 | Day 179 – 180

重点：求职准备

- Day 179：写简历项目描述；整理 GPU / CUDA / AI Infra 技能栈。
- Day 180：最终考试：能解释 GPU、Memory、Warp、Performance、CUDA、AI Kernel、PyTorch Extension、Triton、Nsight，并展示至少 2 个完整 GitHub 项目。

180 天 GitHub 作品集目标

- Repo 1：cuda-kernel-lab
Vector Add / Reduction / Transpose / Softmax / LayerNorm / RMSNorm / Matmul；每个项目必须有代码、Benchmark、Profiler 分析。
- Repo 2：cuda-vs-triton
同一批 AI Kernel 用 PyTorch、CUDA、Triton 实现并比较 latency、throughput、accuracy。
- Repo 3：flash-attention-from-scratch
从 Naive Attention → Tiled Attention → Online Softmax → FlashAttention，记录每次优化。
- Repo 4：pytorch-cuda-extension
把自写 CUDA Operator 正确注册并接入 PyTorch，包含测试与 Benchmark。
- Repo 5：gpu-performance-notes
记录 GPU Architecture、Memory、Warp、Nsight、Tensor Core、Triton、Inference 等长期笔记。

每天固定节奏

时间	任务
45 分钟	理论：官方文档 / 源码 / 教程
90 分钟	编码：当天 Kernel / C++ / PyTorch 项目
30 分钟	Benchmark / Nsight / 性能分析
15 分钟	笔记 + Git commit

三个生死节点

- Day 30：如果 C++ 仍然完全看不懂，先补 C++，不要急着进 CUDA。
- Day 90：如果会写 CUDA 但不知道为什么快/慢，继续 Nsight + Memory + Warp + Optimization，不要急着进入 Triton。
- Day 150：如果 CUDA/Triton 都会一点但没有完整项目，停止扩展新技术，集中完成 FlashAttention。

硬件建议

你现有 RX 5700 XT 可以用于前期 C++、Linux、Git、GPU 原理和部分 Python/Triton 学习，但 CUDA 实机阶段需要 NVIDIA GPU。建议 Day 1 – 60 先学习和建立项目，确认自己愿意长期投入后，再考虑二手 NVIDIA GPU 或云 GPU。不要为了开始学习而立刻购买高价硬件。

180 天之后

下一阶段可以继续：CUTLASS / Tensor Core → FlashInfer / vLLM → KV Cache / Quantization → NCCL / Multi-GPU → LLM Inference → GPU / AI Systems 求职。

第一天现在就做

1. 安装 Git、VS Code、CMake、C++。
2. 创建 GitHub 仓库：cuda-career-180。
3. 写 hello.cpp。
4. 写 README，记录目标与 Day 1。
5. 完成第一次 git commit。
6. 不要急着装 CUDA，也不要同时学 TensorFlow、PyTorch、大模型。